Building an AI-Powered Presentation Generator

Welcome to an exciting journey into building an AI system that automatically creates engaging presentations. We'll explore how Python and cutting-edge AI can transform a simple topic into a structured, visually appealing presentation.

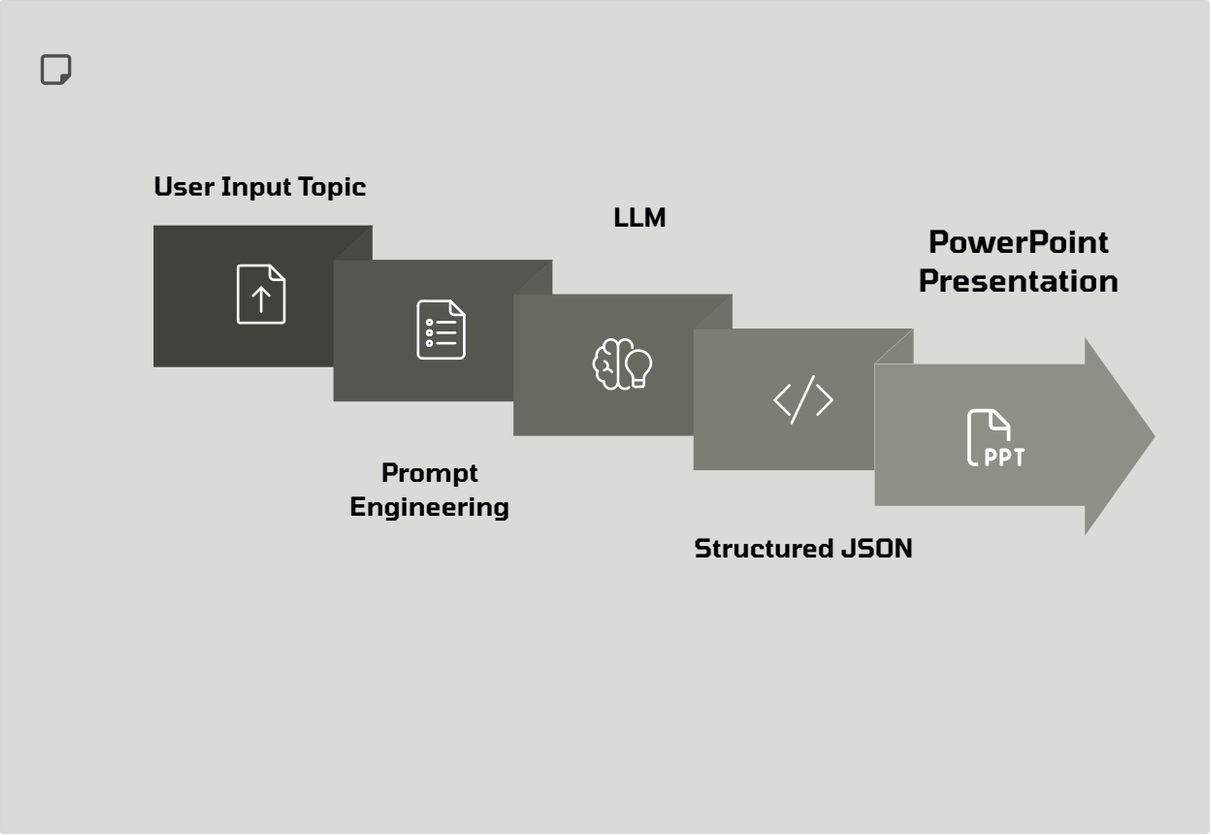


Table of contents

- What Are We Building?
- Core Technologies at Play
- API Keys & Authentication
- Calling APIs in Python
- How to Create a Gemini API Key
- How to create an API splash key
- Continued ...
- PowerPoint in python
- Create a powerpoint with python
- Further powepoint slides with python
- Understanding JSON
- Why AI Must Output JSON
- Using the Built-in JSON Library in Python
- JSON as a Contract
- Mastering Prompt Engineering
- How to create a prompt
- Continued...
- Streamlit library in python
- Streamlit Tutorial (From Zero to Working UI)
- Continued...
- Learning Outcomes

What Are We Building?

User Input

The journey begins when a user provides a topic or query for their presentation.

AI Content Generation

Our AI then processes the topic, generating structured content for each slide.

Python File Creation

Finally, Python takes this content and generates a complete PowerPoint file.

Student Task:

Write the system flow using arrows: User → ? → ? → PowerPoint.

Core Technologies at Play

1	**AI (Gemini) API** Leveraged for intelligent content generation and structuring.
2	**Image Generation (Unsplash) API** Provides stunning, high-quality images to visually enhance slides.
3	**Python** The backbone, orchestrating all components and logic.
4	**JSON** The structured data format for AI output and inter-component communication.
5	**python-pptx** A library for programmatically creating and modifying PowerPoint files.
6	**Streamlit** Used to build a simple, interactive web user interface for the system.

Student Task:

Match each technology to its primary role in the system.

API Keys & Authentication

An API (Application Programming Interface) acts as a messenger, allowing different software applications to communicate. API keys are unique identifiers that authenticate requests, ensuring only authorized applications can access the service.

Gemini API Key

Authenticates your requests to Google's AI models for content generation.

Unsplash API Key

Authorizes access to Unsplash's vast image library for visual assets.

Student Task:

Why can't we call an API without an API key?

Calling APIs in Python

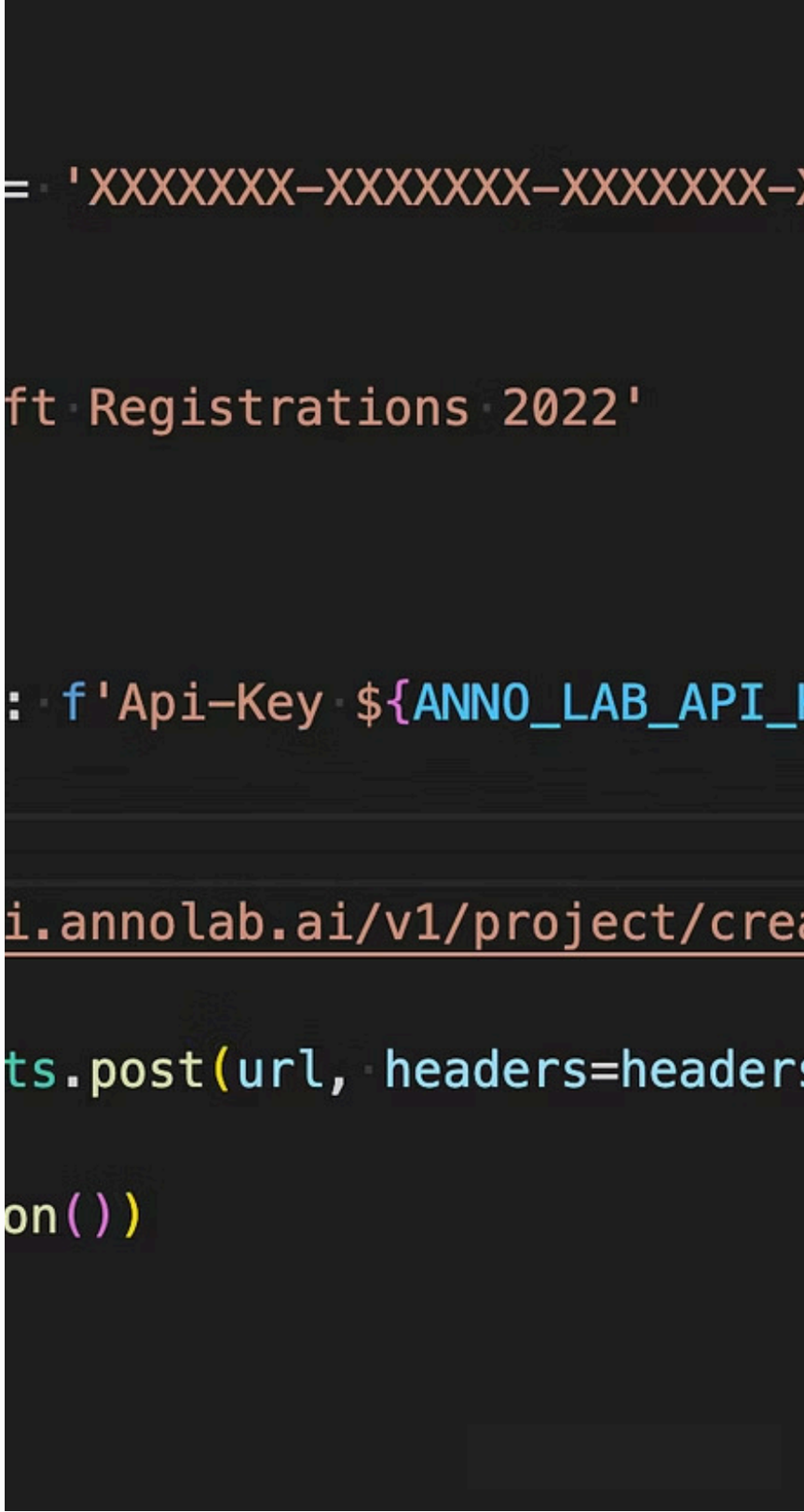
Python uses libraries like `requests` to make HTTP requests to external services. These requests send data (like your API key and query) and receive responses, typically in JSON format.

```
import requests

def call_gemini_api(prompt, api_key):
    headers = {"Authorization": f"Bearer {api_key}"}
    payload = {"text": prompt}
    response = requests.post("https://api.gemini.ai/generate",
json=payload, headers=headers)
    return response.json()
```

Student Task:

Given the function above, identify what is sent to the API and what is received.



How to Create a Gemini API Key

Step 1 – Open Google AI Studio

- Go to **Google AI Studio**
- Sign in with your Google account

Step 2 – Create an API Key

- Click **"Get API Key"**
- Choose **"Create API Key"**
- Select a project (or create a new one)

Step 3 – Copy Your API Key

- Copy the generated key
- Keep it **private** (do not share it publicly)

Step 4 – Use It in Python

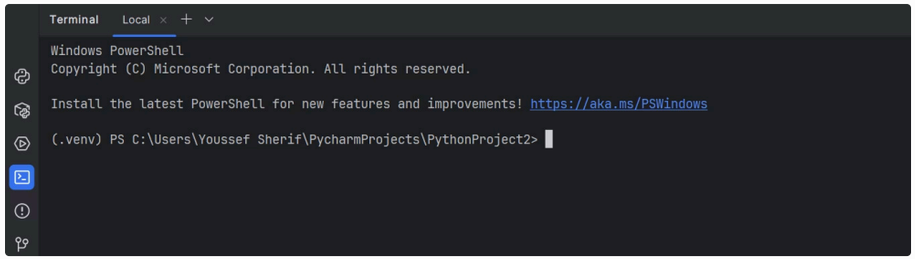
```
import google.generativeai as genai

genai.configure(api_key="YOUR_API_KEY_HERE")
```

Required Python Libraries (Terminal Setup)

Before running the project, install the required libraries using **pip**.

Step 1 – Open Terminal / Command Prompt



Step 2 – Install Required Packages

```
pip install google-generativeai
```

How to create an API splash key

Step 1 – Go to Unsplash Developers

1. Open your browser
2. Go to:
   👉 **https://unsplash.com/developers**

Explain to students:

> Unsplash provides free, high-quality images through an API.

Step 2 – Create / Log in to an Unsplash Account

1. Click **Login** (top right)
2. Log in using:
   - Google account **OR**
   - Email & password

📌 Important:

- You **must be logged in** to create an API key.

Step 3 – Confirm Your Email (IMPORTANT)

1. Check your email inbox
2. Click the **confirmation link**
3. Make sure your account status shows **"Account Confirmed"**

⚠️ Tell students:

> Without email confirmation, the API key will NOT work.

Step 4 – Register as a Developer

1. Go again to:
   👉 **https://unsplash.com/developers**
2. Click **Register as a developer**
3. Accept the terms

Explain:

> This step allows Unsplash to track API usage.

Continued ...

Step 5 – Create a New Application

1. Go to **Applications**
2. Click **New Application**
3. Accept the API guidelines
4. Fill in:
   - **Application name:**
     Example: `AI PowerPoint Generator`
   - **Description:**
     Example: `Educational project using AI to create slides`
   - **Use case:**
     Select *Demo / Educational*

✔️ Click **Create Application**

Step 6 – Copy Your Access Key

After creating the app, you will see:

- **Access Key** ✅ ← THIS IS WHAT YOU NEED
- Secret Key (❌ do NOT use in frontend apps)

📌 Tell students clearly:

> We only use the **Access Key**, not the Secret Key.

Step 7 – Use the Key in Python

Example code:

```
UNSPLASH_ACCESS_KEY = "YOUR_ACCESS_KEY_HERE"
```

Then in API calls:

```
headers = {
    "Authorization": f"Client-ID {UNSPLASH_ACCESS_KEY}"
}
```

Step 8 – Test That It Works (Optional)

Quick test using Python:

```
import requests

url = "https://api.unsplash.com/search/photos"
headers = {
    "Authorization": "Client-ID YOUR_ACCESS_KEY"
}
params = {
    "query": "technology",
    "per_page": 1
}

response = requests.get(url, headers=headers,
params=params)
print(response.json()["results"][0]["urls"]["regular"])
```

If a URL prints → ✅ **API key works**

PowerPoint in python

PowerPoint files can be created using libraries installed. Then you can "render" the part and slides as you want through lines of code

Create a powerpoint with python

✅ Required library

```
pip install python-pptx
```

Why?

- This library is **the rendering engine**
- It creates slides, text boxes, shapes, images, fonts
- It has **nothing to do with AI**

🎯 Goal

Understand **what actually creates a** `.pptx` **file**.

Code (RUN THIS FIRST)

```
from pptx import Presentation

prs = Presentation()
prs.save("step1_empty.pptx")

print("Empty PowerPoint created")
```

Further powepoint slides with python

STEP 2 — Create ONE Slide

🎯 Goal

Understand **how slides are created**

```
from pptx import Presentation

prs = Presentation()

slide = prs.slides.add_slide(prs.slide_layouts[1])
slide.shapes.title.text = "My First Slide"

prs.save("step2_one_slide.pptx")
```

🔴 CORE LINE

```
slide = prs.slides.add_slide(prs.slide_layouts[1])
```

STEP 3 — Add Bullet Points

🎯 Goal

Understand **text placeholders**

```
from pptx import Presentation

prs = Presentation()
slide = prs.slides.add_slide(prs.slide_layouts[1])

slide.shapes.title.text = "What is JSON?"

body = slide.placeholders[1]
body.text = "Structured data\nEasy to read\nUsed by machines"

prs.save("step3_bullets.pptx")
```

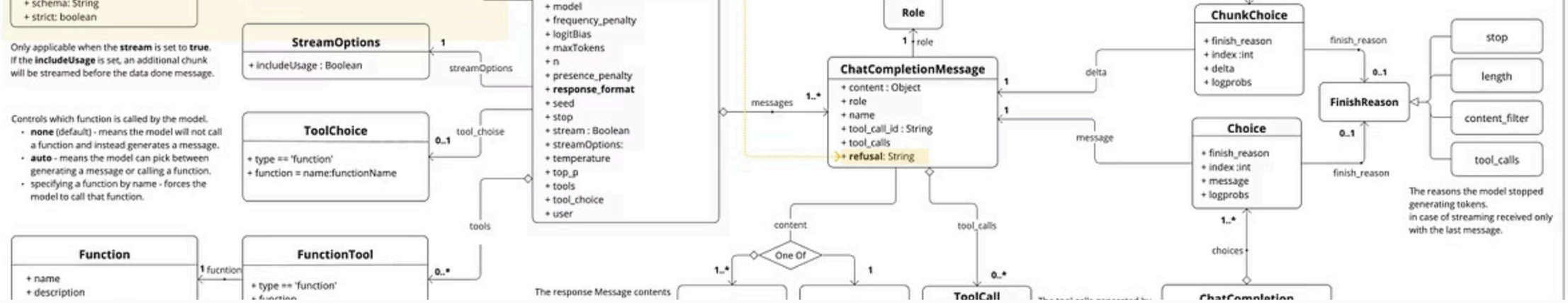
Understanding JSON

JSON (JavaScript Object Notation) is a lightweight data-interchange format. It's human-readable and easy for machines to parse. JSON structures data using key-value pairs and ordered lists, making it ideal for transferring information between our AI and Python code.

```
{
  "title": "Introduction to AI",
  "slides": [
    {
      "heading": "What is AI?",
      "content": ["Definition of AI", "History of AI"]
    },
    {
      "heading": "Applications",
      "content": ["Healthcare", "Finance", "Education"]
    }
  ]
}
```

Student Task:

Given the JSON object above, identify the presentation title, the number of slides, and the bullet points for the "Applications" slide.



Why AI Must Output JSON

While AI can generate free-form text, Python code struggles to reliably extract specific structures from it. By forcing the AI to output in JSON, we ensure the data is consistently organized. This structured format makes it predictable and easy for our Python script to parse and use, eliminating ambiguity.

AI Output: Text	AI Output: JSON
Unpredictable, difficult to parse programmatically.	Structured, consistent, easily consumable by code.

Student Task:

Why is JSON easier for Python to work with than normal text?

Using the Built-in JSON Library in Python

1️⃣Importing the Library

```
import json
```

- Built-in Python module
- No installation needed
- Used to convert between JSON and Python objects

2️⃣load()

Read JSON from a File → Python Object

```
import json

with open("data.json", "r") as file:
    data = json.load(file)

print(data)
```

- Reads JSON string
- Converts to Python dictionary

3️⃣ loads()

Read JSON from a String → Python Object

```
import json

text = '{"name": "Youssef", "age": 22}'
data = json.loads(text)

print(data)
```

- Reads JSON string
- Converts to Python dictionary

4️⃣ dump()

Write Python Object → JSON File

```
import json

data = {"name": "Youssef", "age": 22}

with open("data.json", "w") as file:
    json.dump(data, file)
```

- Saves Python dictionary
- Creates JSON file

5️⃣ dumps()

Convert Python Object → JSON String

```
import json

data = {"name": "Youssef", "age": 22}
text = json.dumps(data)

print(text)
```

- Converts dictionary
- Returns JSON formatted string

JSON as a Contract

Think of JSON as a formal contract between the AI and our Python application. The AI's role is to decide *what* content to generate, adhering to the specified JSON structure. Our Python code's role is to decide *how* to take that structured content and render it into a presentation, applying design and layout rules.

This separation of concerns ensures a robust and scalable system.

Mastering Prompt Engineering

A prompt is the instruction or query given to an AI model. Clear, specific prompts are crucial because they directly control the AI's output. Well-engineered prompts guide the AI to generate relevant, structured, and high-quality content.

> "The quality of the output depends on the quality of the input."

Student Task:

Improve this prompt for our system: "Make slides about AI."

How to create a prompt

Step 1: Define the AI's ROLE (VERY IMPORTANT)

Why?

If you don't define a role, the AI behaves randomly.

Example from your prompt

> You are a professional pitch-deck designer.

Step 2: Clearly State the TASK

Rule:

One task. One clear sentence.

Example

> Create a 5-slide PowerPoint about: {topic}

Why this matters:

- Number of slides is fixed
- Topic is injected dynamically
- No ambiguity

Step 3: FORCE the OUTPUT FORMAT (CRITICAL STEP)

Tell students this sentence (very important):

> AI normally talks to humans.
> We must force it to talk to **Python**.

Example

> Return ONLY valid JSON:

Explain:

- "ONLY" removes explanations
- "valid JSON" removes free text

Practice Question

❓ What happens if we remove the word **ONLY**?

(Expected: AI adds explanations → JSON breaks)

Step 4: SHOW the JSON STRUCTURE (THE CONTRACT)

This part is **not data**, it is a **template**.

Example

```
{
  "title": "Deck title",
  "theme": {
    "background": "light or dark",
    "primary_color": "#HEX",
    "accent_color": "#HEX",
    "font": "Calibri",
    "design_style": "card | split | minimal"
  }
}
```

Explain:

- Keys = what Python expects
- Values = what AI decides

Key teaching sentence:

> JSON is a contract between AI and code.

Continued...

Step 5: Control DESIGN via JSON (Advanced but Powerful)

Explain this clearly:

- AI does NOT draw slides
- AI only **decides design parameters**
- Python applies them

Example

```
"design_style": "card | split | minimal"
```

Explain:

- AI chooses one
- Python checks it
- Python renders it

Step 6: Describe SLIDE CONTENT as DATA

Example

```
"slides": [
  {
    "title": "Slide title",
    "bullets": ["Point 1", "Point 2"],
    "image_query": "professional photo keywords",
    "layout": "text_left_image_right"
  }
]
```

Explain:

- One object = one slide
- Lists allow looping
- Keys map directly to code

Step 7: Add STRICT RULES (GUARDRAILS)

Example

```
Rules:
- Exactly 5 slides
- No markdown
- No explanation
```

Explain:

- Rules reduce randomness
- They protect your parser
- They increase reliability

Streamlit library in python

Streamlit Tutorial (From Zero to Working UI)

What is Streamlit? (Explain First)

Streamlit is a Python library that allows us to:

- Create **web apps**
- Using **only Python**
- Without HTML, CSS, or JavaScript

💡 In our project:

> Streamlit = user interface for entering the topic and downloading the PowerPoint.

🔧 Step 1 – Install Required Libraries

Open **Terminal / Command Prompt** and run:

```
pip install streamlit
```

Step 2 – Your First Streamlit App

Create a file:

```
app.py
```

Write this code:

```
import streamlit as st

st.title("My First Streamlit App")
st.write("Hello, Streamlit!")
```

Run it using:

```
streamlit run app.py
```

🌍 A browser opens automatically.

Continued...

✏️ Step 3 – Taking User Input

```
import streamlit as st

st.title("AI PowerPoint Generator")

topic = st.text_input("Enter your presentation topic")

if topic:
    st.write("You entered:", topic)
```

Explain:

- `text_input()` → input box
- `if topic:` → runs only after typing

🔘 Step 4 – Buttons (User Action)

```
if st.button("Generate Presentation"):
    st.success("Generating your PowerPoint...")
```

Explain:

> Nothing happens until the user clicks the button.

Learning Outcomes

1

AI Systems

Understanding end-to-end AI application development.

2

API Integration

Connecting with external services and data sources.

3

Prompt Engineering

Crafting effective instructions for AI models.

4

JSON & Data Structure

Mastering structured data for seamless inter-component communication.

5

Rendering Engines

Programmatically generating documents and visual content.

6

UI Development

Building user-friendly interfaces for complex systems.

Student Task:

Which part of this project felt most challenging to you and why?